MARKOV MODELS FOR TEXT PROCESSING

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Implement a Markov model to calculate state-transition probabilities and predict the next state or word in a sequence.

1. What is a Markov Model?

A Markov Model is a mathematical model used to predict the next state based on the current state. In NLP, each word can be treated as a state. For example, in “I love machine learning”, the word sequence can be represented as transitions: I → love → machine → learning.

The **first-order Markov assumption** states that the next state depends only on the current state, not the complete history of earlier states:

$$P(X_{t+1} \mid X_t, X_{t-1}, \ldots) = P(X_{t+1} \mid X_t)$$

2. How does the program work?

The implementation follows five main steps: create a text corpus; tokenize the sentences; count state transitions; calculate transition probabilities; and predict the next word.

3. Complete Python program

Create a new file named **markov_model.py** and enter the following code:

```
from collections import defaultdict, Counter

# STEP 1: Create a sample text corpus
corpus = [
    "I love machine learning",
    "I love deep learning",
    "I study machine learning",
    "I study artificial intelligence",
    "Machine learning is interesting"
]

# STEP 2: Tokenization
tokenized_corpus = []

for sentence in corpus:
    words = sentence.lower().split()
    tokenized_corpus.append(words)

print("TOKENIZED CORPUS:")
for sentence in tokenized_corpus:
    print(sentence)


# STEP 3: Count state transitions
transition_counts = defaultdict(Counter)

for sentence in tokenized_corpus:
    for i in range(len(sentence) - 1):
        current_state = sentence[i]
        next_state = sentence[i + 1]
        transition_counts[current_state][next_state] += 1


# STEP 4: Calculate transition probabilities
transition_probabilities = {}

for current_state, next_states in transition_counts.items():
    total = sum(next_states.values())
    transition_probabilities[current_state] = {}

    for next_state, count in next_states.items():
```

```
        probability = count / total
        transition_probabilities[current_state][next_state] = probability


# STEP 5: Display transition probabilities
print("\nSTATE TRANSITION PROBABILITIES:")

for current_state, next_states in transition_probabilities.items():
    print("\nCurrent State:", current_state)

    for next_state, probability in next_states.items():
        print("Next State:", next_state,
              "Probability:", round(probability, 2))


# STEP 6: Predict the next word
def predict_next_word(current_word):
    current_word = current_word.lower()

    if current_word not in transition_probabilities:
        print("\nNo transition information available for:", current_word)
        return

    next_states = transition_probabilities[current_word]
    predicted_word = max(next_states, key=next_states.get)

    print("\nCurrent Word:", current_word)
    print("Predicted Next Word:", predicted_word)
    print("Transition Probability:",
          round(next_states[predicted_word], 2))


# STEP 7: Test the Markov Model
predict_next_word("i")
predict_next_word("love")
predict_next_word("machine")
```

4. Explanation of each step

Step 1 — Create a corpus

A corpus is a collection of sentences. The program learns word transitions from the five sample sentences. It learns only from the supplied corpus.

Step 2 — Tokenization

```
for sentence in corpus:
    words = sentence.lower().split()
    tokenized_corpus.append(words)
```

lower() converts text to lowercase. **split()** separates a sentence into individual words. For example, “I love machine learning” becomes ['i', 'love', 'machine', 'learning'].

Step 3 — Count state transitions

```
transition_counts = defaultdict(Counter)

for sentence in tokenized_corpus:
    for i in range(len(sentence) - 1):
        current_state = sentence[i]
        next_state = sentence[i + 1]
        transition_counts[current_state][next_state] += 1
```

The program examines each pair of consecutive words. The first word is the **current state**, and the following word is the **next state**. The counter records how often each transition occurs.

Current state	Next state	Frequency
i	love	2
i	study	2
love	machine	1

Current state	Next state	Frequency
love	deep	1
machine	learning	3
study	machine	1
study	artificial	1
deep	learning	1
artificial	intelligence	1
learning	is	1
is	interesting	1

Step 4 — Calculate transition probabilities

The transition probability is calculated as:

$$P(B \mid A) = \text{Count}(A \rightarrow B) / \text{Total transitions from } A$$

Example: the word “i” is followed by “love” twice and by “study” twice. There are four transitions from “i”. Therefore P(love | i) = 2/4 = 0.5 and P(study | i) = 2/4 = 0.5. The outgoing probabilities from a state add up to 1.

Transition from i	Probability	Percentage
i → love	0.50	50%
i → study	0.50	50%
Total	1.00	100%

Step 5 — Predict the next word

```
next_states = transition_probabilities[current_word]
predicted_word = max(next_states, key=next_states.get)
```

The max() function selects the next word with the highest transition probability. For example, after “machine”, the only observed next word is “learning”, with probability 1.0.

5. Expected output

```
TOKENIZED CORPUS:
['i', 'love', 'machine', 'learning']
['i', 'love', 'deep', 'learning']
['i', 'study', 'machine', 'learning']
['i', 'study', 'artificial', 'intelligence']
['machine', 'learning', 'is', 'interesting']

STATE TRANSITION PROBABILITIES:

Current State: i
Next State: love Probability: 0.5
Next State: study Probability: 0.5

Current State: love
Next State: machine Probability: 0.5
Next State: deep Probability: 0.5

Current State: machine
Next State: learning Probability: 1.0

Current State: learning
Next State: is Probability: 1.0

Current State: study
Next State: machine Probability: 0.5
Next State: artificial Probability: 0.5

Current State: deep
```

```
Next State: learning Probability: 1.0

Current State: artificial
Next State: intelligence Probability: 1.0

Current State: is
Next State: interesting Probability: 1.0

Current Word: i
Predicted Next Word: love
Transition Probability: 0.5

Current Word: love
Predicted Next Word: machine
Transition Probability: 0.5

Current Word: machine
Predicted Next Word: learning
Transition Probability: 1.0
```

Note: If two next words have equal probabilities, max() selects the first one encountered in the dictionary.

6. How to execute the program

1. Open IDLE, VS Code, PyCharm or Jupyter Notebook.
2. Create a Python file named markov_model.py.
3. Paste the complete program and save it.
4. Open Command Prompt or the terminal in the file's folder.
5. Run the command below:

```
python markov_model.py
```

No external Python libraries are required. The program uses Python's built-in collections module.

7. Experiment with different inputs

```
predict_next_word("study")
predict_next_word("deep")
predict_next_word("artificial")
predict_next_word("learning")
```

Change the test words and observe how the predicted next word changes. If a word does not occur as a current state with an outgoing transition, the program reports that no transition information is available.

8. Markov Model versus N-gram Language Model

Markov Model	N-gram Model
Models transitions between states.	Models sequences of N words.
In a first-order model, the next state depends on the current state.	The next word depends on the previous N–1 words.
Can represent word transitions.	Includes Unigram, Bigram and Trigram models.

9. Applications of Markov Models in NLP

• Next-word prediction
• Text generation
• Speech recognition
• Part-of-speech tagging
• Spell checking
• Predictive text systems

10. Student exercise

Question 1: What is the probability of “study” occurring after “i”? **Answer:** 0.5, because the transition occurs twice out of four transitions from “i”.

Question 2: What is the predicted next word after “machine”? **Answer:** learning, with probability 1.0.

Question 3: What does the Markov assumption mean? **Answer:** In a first-order model, the next state depends only on the current state.

Expected laboratory outcome: Students should be able to construct a first-order Markov model, calculate state-transition probabilities, and predict the next word based on the current word.